Adam Ibrahim

CSC 380

Dave Toth

# ChessOS

I built a bare-metal operating system that boots from a floppy disk image in QEMU. Based on top of the Project A-D from class. The base OS is a working kernel with a file system and shell, on top of that I added a two-player chess game, basic shell, simulators for process scheduling and memory allocation, and a timer interrupt handler with a live clock at the top right.

**Contents:**

- bootloader, kernel, custom INT 0x21 system calls
- file system on the floppy: read, write, copy, delete, rename, create
- chess-themed boot splash painted directly to video memory
- two-player chess game (chess.c)
- simulated process scheduler with FCFS / SJF / Round Robin / Priority and a compare scheduler command that shows all four side by side
- simulated memory allocator with first-fit / best-fit / worst-fit and a fragmentation bar
- live Hour:Minute:Seconds clock in the top-right corner
- uptime, sleep, time <cmd> shell commands

**How to compile:**

You need bcc, as86, ld86, nasm, gcc, and qemu-system-x86 installed.

Then, from the ChessOS folder, run:

./compileOS.sh

**How to run:**

qemu-system-x86_64 -display curses -drive format=raw,file=floppya.img,index=0,if=floppy

**How to use:**

Boot up. Then, type 'help'. It will provide you with a list of commands and brief descriptions. Feel free to play around and break it!